Elide Events – Use case diagram (v7)

System: Elide Events

«package» Account and event search

- Register
- Log in
- Log out
- Search published events
- View event details
- Apply as organizer
- Send email (Register «include»)
- Show location on map (View event details «include»)

«package» Booking, payment, tickets

- Reserve tickets (10-min hold)
- Release seats on expiry (30 s job) («extend»)
- Select seats on the hall map («include»)
- Reject reservation (seat taken / over 8 tickets) («extend»)
- Reject payment: hold expired («extend»)
- Pay order
- Check hold before charging the card («include»)
- Process payment via Stripe (webhook) («include»)
- Handle failed payment («extend»)
- Issue tickets with QR token («extend»)
- Send email («include»)
- View tickets

«package» Organizer: events and subscription

- Create / edit event (DRAFT)
- Geocode address («include»)
- Delete event (only with no tickets sold)
- Reduce ticket category capacity
- Manage ticket types
- Cancel event (compensation flow)
- Start refunds (Stripe, future sprint) («include»)
- Configure hall and seats
- Publish event
- Check approved organizer («include»)
- Validate ticket by QR code
- Reject invalid, used or wrong-event ticket («extend»)
- Sales reports and statistics
- Activate subscription (FREE_TRIAL / MONTHLY)
- Process subscription payment («include»)

«package» Platform administration

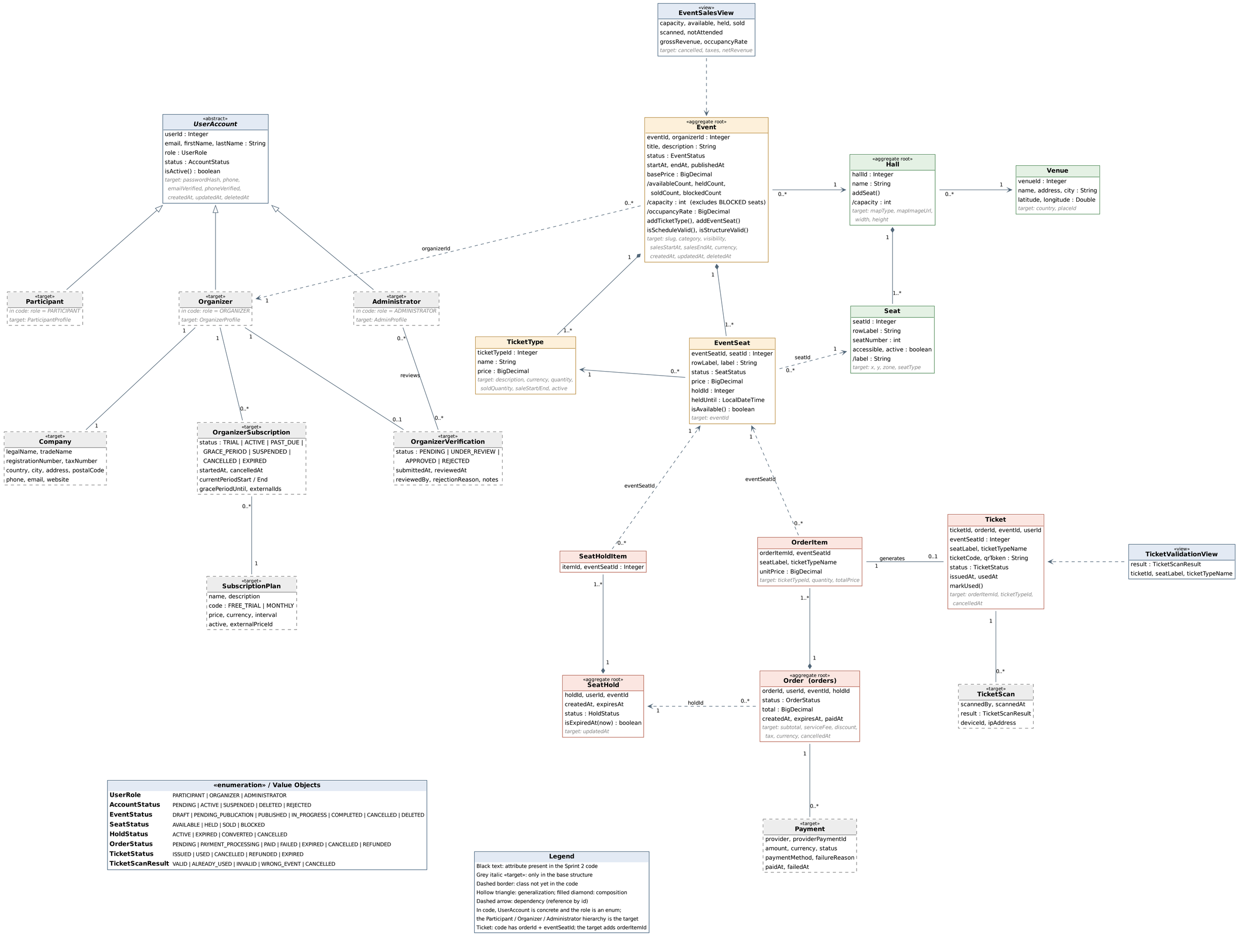
- Approve / reject organizer
- Manage users and permissions (RBAC)
- Monitoring: KPIs, subscriptions, audit

Actors

- User (authenticated)
- Participant
- Organizer
- Administrator
- SendGrid «external system»
- Google Maps Platform «external system»
- Scheduler (periodic job)
- Stripe (Test Mode + Billing) «external system»

External actors are repeated only for readability (same system).
Include / extend link only use cases of the same package; “Send email” appears in every package that uses it.
Payment: startPayment checks the hold BEFORE charging; if it expired the payment is rejected and the card is not charged.
Issue tickets (extend): payment is confirmed by Stripe webhook, not by the browser.
Release seats (extend): 10 minutes elapsed, but not while a started payment is in progress (5 min grace).
Reservation: at most 8 tickets per user per event (TicketLimitExceededException); seat taken = 409.
Cancel: allowed before COMPLETED even with sold tickets; delete: only with no tickets sold.
A PENDING organizer may create a DRAFT; publishing needs an approved (ACTIVE) account. Code has approval only; rejection is a target.
Sprint 2 code: every event has a hall and seats, so seat selection is «include».
QR validation is done by the organizer (TICKET_VALIDATE); no separate actor.

Elide Events – Domain class diagram (v7)



«enumeration» / Value Objects	
UserRole	PARTICIPANT | ORGANIZER | ADMINISTRATOR
AccountStatus	PENDING | ACTIVE | SUSPENDED | DELETED | REJECTED
EventStatus	DRAFT | PENDING_PUBLICATION | PUBLISHED | IN_PROGRESS | COMPLETED | CANCELLED | DELETED
SeatStatus	AVAILABLE | HELD | SOLD | BLOCKED
HoldStatus	ACTIVE | EXPIRED | CONVERTED | CANCELLED
OrderStatus	PENDING | PAYMENT_PROCESSING | PAID | FAILED | EXPIRED | CANCELLED | REFUNDED
TicketStatus	ISSUED | USED | CANCELLED | REFUNDED | EXPIRED
TicketScanResult	VALID | ALREADY_USED | INVALID | WRONG_EVENT | CANCELLED

Legend

- Black text: attribute present in the Sprint 2 code
- Grey italic «target»: only in the base structure
- Dashed border: class not yet in the code
- Hollow triangle: generalization; filled diamond: composition
- Dashed arrow: dependency (reference by id)
- In code, UserAccount is concrete and the role is an enum; the Participant / Organizer / Administrator hierarchy is the target
- Ticket: code has orderId + eventSeatId; the target adds orderItemId

Elide Events – Service layer: workflow and domain services (v7)

«target»
REST controllers
future sprint (POST /events/{id}/cancel,
DELETE /events/{id}, ...)

«interface / Facade»
IEventTicketingBusinessWorkflowService
approveOrganizer(adminId, organizerId) : Integer
createDraftEvent(organizerId, hallId, title, ...) : Integer
publishEvent(eventId) : Integer
reduceCategoryCapacity(eventId, ticketTypeName, newCapacity) : Integer
rescheduleEvent(eventId, newStartAt, newEndAt) : Integer
cancelEvent(eventId) : List<Integer>
deleteEvent(eventId) : Integer
holdSeats(userId, eventId, eventSeatIds) : Integer
createOrder(holdId) : Integer
startPayment(orderId) : Integer
confirmPayment(orderId) : List<Integer>
failPayment(orderId) : Integer
expireHolds(now) : int
validateTicket(qrToken, eventId) : TicketValidationView
getEventSalesReport(eventId) : EventSalesView

«Factory»
IEventEntityFactory
buildUser(...) buildVenue(...) buildHall(...)
buildEventWithSeats(...) : Event
buildSeatHold(...) buildOrder(...)
buildTicket(...)
Impl: EventEntityFactoryBase

«Domain Service»
IValidatingEventDomainService
validate(Event) : Set<String>
validateWithException(Event)
validateEventAggregate(Event) : boolean
validate(Order) : Set<String>
validateWithException(Order)
Impl: ValidatingEventDomainServiceImpl

«domain package»
Domain model
UserAccount, Event, EventSeat, TicketType,
Hall, Seat, Venue, SeatHold, Order,
OrderItem, Ticket, EventSalesView,
TicketValidationView

«Domain Service»
ISummarizingEventDomainService
countingSeats(Event) : Event
computingOccupancy(Event) : Event
Impl: SummarizingEventDomainServiceImpl

«Service»
EventTicketingBusinessWorkflowServiceImpl
SEAT_HOLD_MINUTES = 10
MAX_TICKETS_PER_USER = 8
PAYMENT_GRACE_MINUTES = 5
seatLock : Object (sprint 2: synchronized)

«Domain Service»
IPricingOrderDomainService
computingTotal(Order) : Order
Impl: PricingOrderDomainServiceImpl

«interface»
Repository interfaces
IUserAccountRepository, IHallRepository,
IEventRepository, ISeatHoldRepository,
IOrderRepository, ITicketRepository
ITicketRepository: findByQrToken(), findByEventId()

«Repository»
***RepositoryBase**
extends InMemoryEntityRepositoryBase<T>
(in memory; JPA in the persistence sprint)

«throws»

«Domain Service»
IConsolidatingEventSalesViewDomainService
getEventSalesViewDataOf(Event, Collection<Ticket>)
: EventSalesView
Impl: ConsolidatingEventSalesViewDomainServiceImpl

«interface»
IEntityRepository<T>
getNextID() : Integer
getById(Integer) : T
toCollection() : Collection<T>
add(T) : T
remove(T) : boolean
size() : int

BusinessRuleViolationException
extends RuntimeException

SeatNoLongerAvailableException
HTTP 409 (later)

«new»
PaymentRejectedException
PAYMENT_REJECTED
the card is not charged

«new»
TicketLimitExceededException
TICKET_LIMIT_EXCEEDED
max 8 tickets / user / event

DomainValidationException
extends RuntimeException

Notes

Layers: controllers (target) → business services → domain services / factory / repository → domain model.

All workflow operations are serialized with seatLock (sprint 2); with persistence they become transactions with row locking.

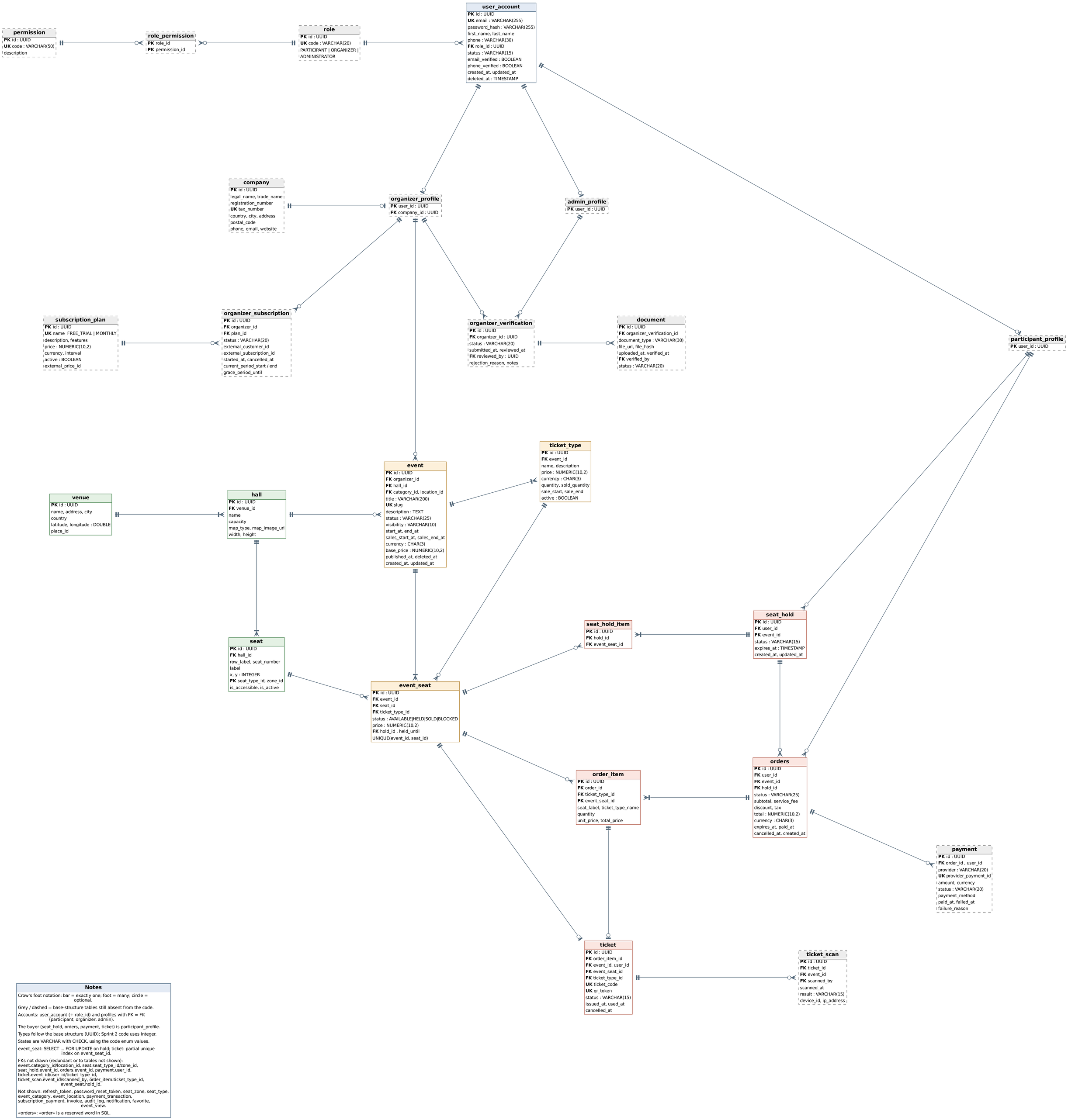
startPayment is called BEFORE charging: expired hold → PaymentRejectedException; otherwise the order becomes PAYMENT_PROCESSING.

expireHolds does not release the seats of a started payment during the grace period (5 min).

cancelEvent: holds released, PENDING orders → CANCELLED, PAID → REFUNDED, ISSUED tickets → REFUNDED; returns the orders to refund.

Event.getCapacity() counts sellable seats only (no BLOCKED); reduceCategoryCapacity blocks surplus AVAILABLE seats.

Elide Events – PostgreSQL database diagram (v7)



Notes

Crow's foot notation: bar = exactly one; foot = many; circle = optional.

Grey / dashed = base-structure tables still absent from the code.

Accounts: user_account (+ role_id) and profiles with PK = FK (participant, organizer, admin).

The buyer (seat_hold, orders, payment, ticket) is participant_profile.

Types follow the base structure (UUID); Sprint 2 code uses Integer.

States are VARCHAR with CHECK, using the code enum values.

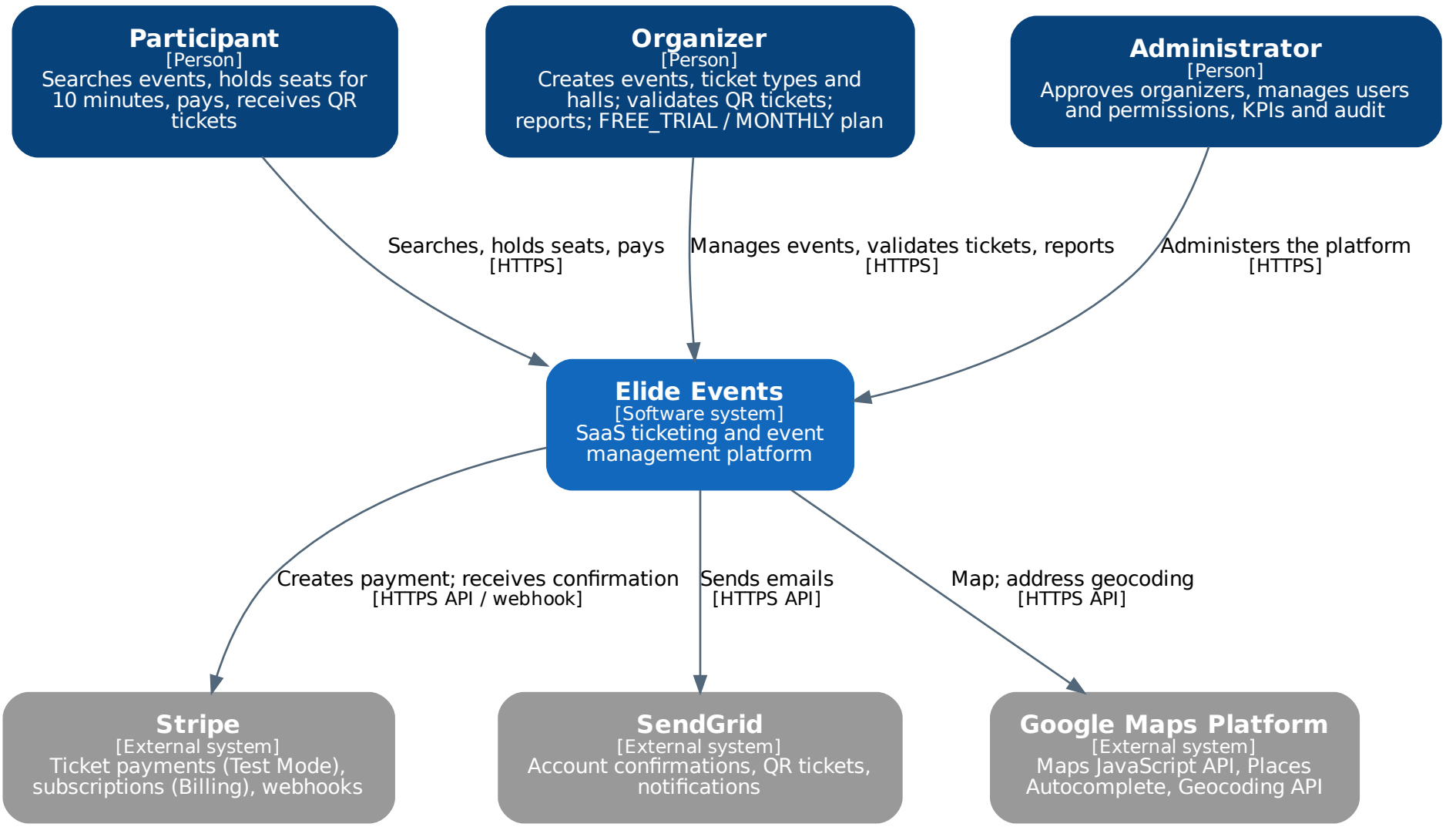
event_seat: SELECT ... FOR UPDATE on hold; ticket: partial unique index on event_seat_id.

FKs not drawn (redundant or to tables not shown): event.category_id/location_id, seat.seat_type_id/zone_id, seat_hold.event_id, orders.event_id, payment.user_id, ticket.event_id/user_id/ticket_type_id, ticket_scan.event_id/scanned_by, order_item.ticket_type_id, event_seat.hold_id.

Not shown: refresh_token, password_reset_token, seat_zone, seat_type, event_category, event_location, payment_transaction, subscription_payment, invoice, audit_log, notification, favorite, event_view.

«orders»: «order» is a reserved word in SQL.

Elide Events – C4 level 1: System context (v7)



Elide Events – C4 level 2: Containers (v7)

